



COURSE CONTENT

COURSE NAME

Frontend Engineer

OVERVIEW

Training	Duration (days)
Time Management	1
HTML5, CSS3, Javascript	5
Typescript	2
SDLC & Agile	2
Design Thinking Basics	1
GIT	2
JIRA	2
Creative Thinking and Problem Solving	2
React with Redux	8
Angular Basics	3
Node JS with RESTful API	4
Jest and Enzyme	3
ESLint for Code Quality	2
Application Security for React Developers	3
React Native Fundamentals	3
Introduction to Generative AI [Virtual]	2
Capstone Project	5
TOTAL	50

DURATION

50 Days

COURSE OUTLINE

Time Management – 1 Day

- Identifying your greatest 'time stealers'
- Guiding principles of time and self-management
- Identifying and managing peak physiological performance time
- Saying 'no' and negotiating solutions
- Prioritising workload and key tasks
- Planning and prioritising your own workload
- Delegating tasks
- Assigning tasks to be delegated

- Solutions for managing time effectively

HTML5, CSS3, Javascript – 5 Days

- HTML Review
- Introduction to HTML5
- CSS Basics
- CSS3 Overview
- CSS3 Advanced Selectors
- CSS3 Visual Effects
- JavaScript Basics
- Debugging Tools
- JavaScript Functions
- JavaScript Arrays, Math and Date
- JavaScript Event Handling and the DOM
- Object-Oriented JavaScript

Typescript – 2 Days

- Introduction to TypeScript
 - What is TypeScript?
 - Benefits of Using TypeScript
 - TypeScript vs JavaScript
- Getting Started with TypeScript
 - Installing TypeScript
 - Setting Up the Development Environment (Visual Studio Code)
 - First TypeScript Program
- Basic Types
 - Type Annotations
 - Primitive Types (number, string, boolean)
 - Arrays and Tuples
 - Enum Types
- Functions
 - Function Types
 - Optional and Default Parameters
 - Rest Parameters
 - Arrow Functions
- Interfaces
 - Introduction to Interfaces
 - Optional Properties
 - Readonly Properties

- Function Types and Indexable Types
- **Classes and Objects**
 - Class Basics
 - Inheritance
 - Access Modifiers (public, private, protected)
 - Read only Modifier
 - Static Properties and Methods
- **Hands-on Lab: Basic TypeScript Exercises**
 - Setting Up TypeScript Projects
 - Implementing Basic Types and Functions
 - Creating and Using Interfaces and Classes
- **Advanced Types**
 - Union and Intersection Types
 - Type Aliases
 - Type Guards and Type Assertions
 - Nullable Types
- **Generics**
 - Introduction to Generics
 - Generic Functions and Interfaces
 - Generic Classes
 - Constraints in Generics
- **Modules and Namespaces**
 - Introduction to Modules
 - Importing and Exporting Modules
 - Namespaces and Module Resolution
- **Decorators**
 - Introduction to Decorators
 - Class, Method, and Property Decorators
 - Using Decorators in Real-World Applications
- **TypeScript with JavaScript Libraries**
 - Using TypeScript with Popular JavaScript Libraries (e.g., React, Node.js)
 - Type Definitions with Definitely Typed
 - Integrating TypeScript into Existing JavaScript Projects
- **Hands-on Lab: Advanced TypeScript Project**
 - Developing a Simple Application Using Advanced TypeScript Features
 - Integrating TypeScript with a JavaScript Library (e.g., React)
 - Applying Best Practices for TypeScript Development
- **Best Practices and Tips**
 - Code Organization and Project Structure
 - TypeScript Compiler Options
 - Debugging TypeScript Code

- Common Pitfalls and How to Avoid Them

SDLC & Agile – 2 Days

- The SDLC: The Life Cycles
- Overview of SDLC
- Overview of Project Management
- Overview of Business Analysis
- Agile Overview from Industry Practices
- Introduction to Agile Project Management
- Fundamentals of the Scrum Framework
- Identifying the roles and their responsibilities
- Engineering Practices Overview
- Scaling scrum, SAFe Model Overview

Design Thinking Basics – 1 Day

- Introduction
- Design Thinking: What it is & why it matters
 - What is design thinking
 - Who uses design thinking
 - The value of design thinking
- Empathy: The Foundation of Human-Centered Design
 - Challenge your assumptions
 - Method of Empathy
 - Interviewing techniques
 - Empathy in action
- Define: Making Sense of your data
 - Synthesis Frameworks
 - Personas
 - Point of view (POV) statements
- Ideate: How might we design for our users
 - “How might we...” statements
 - HMW statements: Find and fix the mistake
 - Ideation Tips and Tricks
- Prototypes & Tests: Bring your ideas to life
 - What is prototype
 - Testing a hypothesis
 - Testing as empathy
 - Testing in action

- Make your pitch: Using storytelling to make the case for design thinking
 - The power of storytelling
 - Developing your pitch
 - Pitching in action

GIT – 2 Days

- Introduction
 - About Version Control
 - Centralized vs. Distributed VCSs
 - History of Git
 - Differences between Git and other VCSs
- Configuring Git
 - Installing Git
 - git help
 - git config
 - Setting User Identity
- Working with Git
 - Initializing a Repository in a Directory
 - Cloning an Existing Repository
 - Checking the Status of Files
 - Tracking New Files
 - Staging Modified Files
 - Ignoring Files
 - Moving & Removing Files
 - Working with Remote Repositories
 - Committing Changes
 - Viewing Commit History
 - Rolling Back Changes
- Branching with GIT
 - Branches in Git
 - Creating a Branch
 - Switching Branches
 - Merging Branches
 - Merge Conflicts

JIRA – 2 Days

- Introduction to Jira
 - Getting Started
 - Project Administration
- Jira for Agile Development

- Agile Development
- Managing User Stories, Epics, and Themes
- Product Backlog Management
- Jira in Scrum Processes
 - Scrum Framework Integration
 - Sprint Planning
 - Daily Scrum
 - Tracking Progress: Burndown Charts & Velocity
 - Sprint Retrospective Meetings
 - Sprint Review Practices
- Jira Boards and Reporting
 - Scrum Board
 - Generating Agile Reports
 - Exploring Reports and Dashboards

Creative Thinking and Problem Solving – 2 Days

- Critical Thinking and Creative Problem-Solving Essentials
 - Recognizing business reasons for creative problem solving in the workplace
 - Defining creativity vs. innovation
 - Exploring the thinking process
 - Pinpointing problems and their outcomes
 - Applying outcome-based thinking
- Leveraging Personal Thinking Styles
 - Analyzing personal preferences
 - Assessing your preferred approach to thinking
 - Identifying thinking preferences
- Unleashing Your Creativity
 - The creative environment
 - Group creative thinking
 - The iterative mind
- Solving Problems Using Analysis and Prioritization Tools
 - Applying analysis models
 - Systematic approaches to problem solving
 - Decision analysis
- Translating Creativity and Analysis into Practical Application
 - Identifying problems
 - Avoiding analysis paralysis
 - Deploying your decision
- Putting It All Together
 - Integrating your solution into the business

- Your personal development toolkit

React with Redux – 8 Days

- Introducing React
- Building Your First React App
- Components in React
- Styling React Components
- State & Props
- Transferring Properties (Props)
- Meet JSX—Again!
- Building Content with JSX
- Communicating with Props
- Structuring Apps with Class-Based Components
- State in React Components
- Dealing with State
- Going from Data to UI
- Working with Events
- The Component Lifecycle
- Accessing DOM Elements
- React Router
- Handling User Input with Forms and Events
- AJAX in React
- Building Lists of Records
- Using Ref's for DOM Access
- React Hooks
- Let's Dive In!
- Building Content with JSX
- Communicating with Props
- Structuring Apps with Class-Based Components
- State in React Components
- Understanding Lifecycle Methods
- Handling User Input with Forms and Events
- Making API Requests with React
- Building Lists of Records
- Using Ref's for DOM Access

Angular Basics – 3 Days

- Introduction to Angular
- Angular Fundamentals
- Display Data and Handling Events
- Building Re usable Components
- Directives
- Template Driven forms
- Consuming HTTP Services
- Routing and Navigation
- Wrap Up

Node JS with RESTful API – 4 Days

Introduction

Node.js Concepts

- RAM vs I/O latency
- Blocking vs. non-blocking
- Syntax and logic

The Fundamentals of APIs and Their Functionality

- Scalar types
- Web Architecture Patterns: the composite pattern, proxy pattern, and facade pattern

REST Overview

- Get option
- Pull option
- Post option
- Delete option

Preparing the Development Environment

- Installing and configuring Node.js
- Installing and configuring Express.js
- Installing and configuring MongoDB
- Testing the installations

Node Modules and Package Manager

- Creating a module
- Loading a module
- Using module functions
- Creating event arguments
- Extending event emitters
- Installing a Node package
- Using a package
- Listing packages
- Updating packages
- Uninstalling packages
- Publishing packages

Working with Express.js

- Creating custom middleware
- Using Express router
- Filtering paths

REST and GraphQL API

- Building a web server
- Handling routes
- Parsing HTTP requests
- Calling endpoints
- Defining schema
- Adding input validation
- Managing mutations
- Adding variables
- Handling errors

CRUD Operations Using MongoDB

- Saving documents
- Querying documents
- Updating documents
- Deleting documents

Authentication and Security

- Creating and registering users
- Generating tokens
- Storing in environment variables
- Protecting routes
- Testing the authorization

Troubleshooting

- Writing a unit test
- Writing an integration test
- Wiring a unit and integration test

Summary and Conclusion

Jest and Enzyme – 3 Days

- Setting up Jest without create-react-app
- Enzyme Introduction and Setup?
- Using Enzyme in a Test
- Types of Tests
- Testing Asynchronous Code
- Setup and Teardown
- Mock Functions
- Jest Platform and Community
- Snapshot Testing
- Async Example
- Timer Mocks
- Es6 Class Mocks
- Bypassing Module mocks
- Using with webpack
- using with puppeteer
- using with MongoDB and DynamoDB
- DOM Manipulation
- Watch Plugins
- Migration to Jest and ARchitecture
- Testing React Apps few demo and few custom made apps
- Testing React Native Apps and Web Framework
- In Depth Look in Globals,Expect,Mock Functions,Jest Object and Configuring Jest
- Jest CLI Options

ESLint for Code Quality – 2 Days

Overview

Setting up ESLint in Your Project

- Course Introduction
- Installing and Running ESLint
- Resolving ESLint Errors
- NPM Scripts
- Git Hooks
- Summary

ESLint Rules Explained

- Introduction
- Common ESLint Rules
- Picking and Adding More ESLint Rules
- ESLint Rules Under the Hood
- ESLint and Abstract Syntax Trees (ASTs)
- Writing Your First Rule
- No-FIXME-comment Rule
- Expiring-code Rule
- Passing Parameters to Rules
- Verify-parameters Rule
- Making Rules Fixable with `—fix`
- Summary

Exploring the ESLint Ecosystem

- Introduction
- Shareable Configs and Plugins
- Adding an ESLint Plugin from NPM
- Adding an ESLint Config from NPM
- Creating a Shareable Config
- Creating a Shareable Plugin
- Summary

Common ESLint Use Cases

- Introduction
- Linting Client and Server JavaScript
- Linting ES6/7 JavaScript
- Linting Angular Code
- Linting React Code
- ESLint Editor Integration
- Overriding and Ignoring Files
- Summary

Application Security for React Developers – 3 Days

Introduction

- Securing React Components
- The Attack Surface
- The Threats
- Building Secure React Components
- XSS via a Spoofed React Element

Avoiding Component Injection Attacks

- Introduction to Component Injection Attacks
- Creating React Components
- React's Auto Escaping
- Attacker-Controlled Link Values
- Attacker-Controlled CSS
- Attacker-Controlled Value in Third-Party Components

Avoiding Attacker-Controlled JSON Exploits

- JSON Attacks in the React Ecosystem
- Code Injection via Preloaded State
- Avoiding Attacker-Controlled JSON Attacks
- XSS in the Redux Documentation Example Code

Avoiding Vulnerable Third-Party Libraries

- Reviewing a Module for Code Mistakes
- Reviewing a Module for Insecure Configuration
- Judging Modules Based on Metrics
- Finding Reported Vulnerabilities in React Modules

- Automation
- Backdoor Discovered in Popular Module

Avoiding Trust in Client-Side Routing as a Security Control

- Client-Side Routing
- Security Controls in React Router
- Avoiding Client-Side Enforcement of Server-Side Security

Avoiding Vulnerable Versions of React

- npm audit
- Upgrading React
- XSS in Imgur

React Native Fundamentals – 3 Days

React Basics

- Identify why React was built
- Basics of ES6
- Use composition to build complex functions from simple ones
- Leverage declarative code to express logic without control flow
- Recognize that react is just JavaScript
- Use create-react-app to create a new React application
- Create reusable, focused Class components with composition
- Introduction to JSX and styling
- React Components
- React Components: State and Props
- React Components: Lifecycle Methods
- Third Party Components for React

State Management

- Manage state in applications
- Use props to pass data into a component
- Create functional components focused on UI rather than behavior
- Add state to components to represent mutable internal data
- Use the this keyword to access component data and properties
- Update state with setState()
- Use PropTypes to typecheck and debug components
- Use controlled components to manage input form elements

- Recognize how state predictability improves applications
- Create a store to manage an applications state
- Leverage store API: `getState()`, `dispatch()`, and `subscribe()`
- Create Actions and Action Creators that describe state changes
- Create Reducers that return state
- Use Reducer Composition to handle independent parts of state

Render UI with External Data

- Conceptualize the lifecycle of a component
- Use React's component Did Mount lifecycle hook for HTTP requests

Manage App Location with React Router

- Use React Router to add different routes to applications
- Use state to dynamically render a different "page"
- Use React Router's Route component
- Use React Router's Link component

Redux Middleware

- The Flux Architecture
- React Redux Forms
- Identify the benefits of implementing middleware in applications
- Identify the role of middleware within the Redux cycle
- Apply middleware to a Redux application
- Build your own Redux middleware
- Combine Redux with the popular React library
- Identify when to use component state vs. Redux state
- Learn the pitfall of asynchronous requests in Redux
- Leverage Thunk middleware to support asynchronous requests
- Fetch data from a remote API
- Promises
- Fetch
- Install the react-redux bindings
- Leverage react-redux bindings to extend app functionality
- Use the Provider to pass a store to component trees
- Use `connect()` to access store context set by the Provider
- WebPack 4.x

React Native Basics

- To understand why to use react-native although we have multiple cross platform technologies like IBM MobileFirst, Cordova, Ionic, Titanium Appcelerator etc are available.
- To understand features and tools available to develop react-native application
- React-Native class components and Functional Components with example.
- Class components in detail

Dive deep into React-Native

- To understand folders and files and their usage while developing react-native application, Appellatory etc are available.
- To understand UI components available in react-native
- Debugging react-native application

Advance topics

- Navigation in react-native (React-navigation and react-native-router-flux)
- Calling RESTful API
- Async programming in react-native
- Flexbox
- Props & States

UI Controls

- Flatlist
- Picker, Share , Toast View
- Async Store
- Key Chain
- Communication between Native and React-Native
- Create Android apk file that could be deployed

Offline Storage and Handson

- Offline Storage (Realm/ SQLite database)
- Develop an example
- Evaluation of participants (From online test)

Introduction to Generative AI [Virtual] – 2 Days

- Definition
- How Generative AI works
- Generative AI Model types
- Generative Applications
- Introduction to Large Language Models
- Attention Mechanism
- Transformer Models
- Image Generation
- Image Captioning Models
- Encoder-Decoder Architecture

Capstone Project – 5 Days